

handoff-fanout 架构缺口与半实现陷阱分析

配套 ARCHITECTURE-OVERVIEW.md。本篇专列「功能架构完整 ≠ 可安全运营」的缺口：看着完成实则休眠/空转的半实现陷阱、运维 NFR 缺口、未修的工程加固债、文档漂移。承重论断带 file:line，已实地 read-back 坐实（含一次 grep 假阴自纠：focus-jump ✓ (空格) 漏匹配真实格式 focus-jump: ✓ (冒号)，实际 70 条命中非 0）。

摘要 — 一句话健康判断

派窗/接续/审计的功能主干是健康的、真在跑：去程 focus-jump、回程 spawn-return、retro 闸、codex 审计闸、交付审计机器闸、succession 一次性 token 都已 live 并有日志/磁盘证据。但「可安全长期自托管运营」还差几块，按爆炸半径排序见 §F。最该警惕的不是「没做的功能」，而是三类「看着完成实则不然」：① 一个会反向卸载 live 扩展的 install.sh；② 24GB 运行时状态零应用级备份；③ 多处陈旧注释/文档把已上线说成休眠、或只描述了 1/3 的系统——非技术读者据此易误判系统边界。

A. 半实现陷阱（UI/CLI/代码存在，实际休眠/空转/反向——比没做更危险）

A1. 🟡 去程 FOCUS-JUMP 的「自动派生 --SELF-TASK」尚未激活（OWNER 主诉根因）

- **现象**：中枢派 worker 一步落中枢桌面，依赖派发时带 --self-task <中枢 task>。显式传时：✓ 真 work (router.log 实证 focus-jump: ✓ 直跳 5→7 / 7→8 (spawner=.../handoff-fanout/singlepane/...))。忘传时：身份解析器本应自动补 --self-task，但解析器对当前 backlog (13+ 个陈旧 supervisor_succession sidecar 全 active) 判 **ambiguous** → 返回 **None** → 不补 → 回退 **per-project goto** (落项目桌面 ≠ 中枢实际桌面)。
- **后果**：中枢一忘传 --self-task，worker 静默不去程到中枢桌面 (owner 实测 ff-coord-4 的症状)。fail-open 不报错=更隐蔽。
- **正解（已具备、待激活）**：身份解析器已修「唯一化或失败」(dx_session_role.py _scan_singlepane_supervisor)，dx-spawn 已加自动派生 (dx-spawn-session.sh derive 块)，但激活前置 = 先把 **backlog GC 收敛到唯一 active** (见 A2 + §E)。在此之前的过渡手段：派发时显式传 --self-task (中枢 runbook 已含)。

A2. 🟡 SIDECAR 卫生 GC 无 LAUNCHD 排程、纯手动

- **现象**：`handoff gc-singlepane` (`gc_singlepane.py`) 能清陈旧 `supervisor_succession` sidecar (解析器歧义的根本)，但没有 **launchd** 定时，只在有人手动 `--execute` 时才跑。当前 dry-run 实测 6 个死中枢候选 (且 `--retention-days 0` 会扫到全部 p12-p26 死中枢)。
- **后果**：backlog 静默累积 → 解析器持续 ambiguous → A1 的自动派生永不激活。`--execute` 是高危 (移文件)，有意默认 dry-run + 可逆 quarantine + 活性闸 (liveness 未知→不清)。
- **正解**：owner 在环跑一次 `handoff gc-singlepane --project handoff-fanout --protect <当前中枢> --retention-days 0 --execute` 收敛到唯一 active (quarantine 可逆、且已限规范路径=破坏面已锁)，之后议是否上 launchd 低频排程。

A3. 🔴 INSTALL.SH 第 5 步会反向卸载 LIVE 扩展 (最严重部署陷阱)

- **现象**：`install/install.sh:252-267` 第 5 步注释写「handoff-helper 扩展已废弃 (auto-close 功能下线)」并执行 `code --uninstall-extension dharmaxis.handoff-helper`。但 live 装的 **0.6.0 扩展** 承载的是 singlepane 折叠 + 完整 §6c worktree 回收 + succession 关窗 —— 远不止 autoclose。
- **后果**：任何人跑 `install.sh` (不带 `--no-extension`) 会拆掉正在用的扩展 → 没单栏、没 worktree 回收、没 succession 关窗。install.sh 的世界观停留在约 2 周前。且 install.sh 从不安装 vsix (只卸载)，扩展靠手装。
- **正解**：删除/改写第 5 步为「安装当前 vsix」或显式 no-op；统一 vsix 命名 (现 `handoff-helper-0.4.0.vsix` 陈旧 + `handoff-helper.vsix` 无版本号并存，装错 0.4.0 会静默降级到 pre-§6c)。

A4. 🟡 §6C WORKER 窗口回收：代码就绪、回收 E2E 从未成功

- **现象**：§6c reclaim (`reclaim.py`) producer (watchdog tick) 已接线、代码全活 (PID dead-man / pending 状态机 / 19-reason 枚举)，但 **tick 哨兵驱动**——无 `reclaim_requested` 哨兵的 tick 是 no-op。**实地坐实的精确水位** (勿过度声称也勿冤枉真功能)：① **host_pid 写入侧真跑过**——盘上 21 个真实任务的 `*.host_pid.json` (扩展 activate 即写 pid，注意文件名是 `<task>.host_pid.json` 非裸 `host_pid.json`，用错 glob 会假阴报零)；② **但完整回收 E2E 从未成功**——0 `reclaim_done` / 0 `close_issued` / 当前 0 `reclaim_pending`；唯一 3 次 `reclaim_failed` (2026-06-11) 全是 `ack-timeout`，且属 A-poll pull 改造之前的旧 push 模型失败 (pull 模型正是为修它而建) → **当前 pull 模型下从未有一次成功回收**。
- **后果**：声称「§6c 端到端 LIVE」= 重蹈历史教训 (扩展侧/producer 侧 ≠ E2E)。真闭环须一次「coordinator→扩展 close_issued→PID-ESRCH→worktree 删」的完整 trace，至今无此证据。
- **正解**：下次有真 worker 合并后走一次启用 runbook (`record-head → reclaim_requested → tick 验闸关窗 → PID 验真 → GC`) 留 E2E 证据，再标✅。

A5. 🟡 SUCCESSION AUTOCLOSE OPT-IN 全未拨（有意默认 OFF）

- **现象**：旧中枢窗自动关的 opt-in 三路（`HANDOFF_AUTOCLOSE_ENABLED` env / 全局 sentinel / per-project sentinel）实查全未拨；producer 在 `auto-continue.sh` 内但 launcher 静默 skip。
- **后果**：交棒后旧中枢窗不自动关、owner 手关。这是有意的安全姿态（误关丢 context > 手关麻烦），非缺陷——但意味着「交棒=旧窗仍开着需人关」。
- **正解**：保持 OFF 直到 go-live nonce 校验新扩展 + 稳定期达标（既有 rollout 门槛）。

A6. 🟡 OWNER_ACK_TOKEN 是 SHA256 拼接、非加密（可被同身份伪造）

- **现象**：`audit-override` / owner 豁免 finding 的校验 token = `sha256(task|finding_hash|nonce|approved_at)`（`codex_audit.py:160` / `audit_evidence.py:136`），非加密签名。tty 闸（`audit_evidence.py:441`）只挡「无 tty 的 AI 会话」，不挡「以 owner 身份运行的进程伪造自洽 token」。
- **后果**：单用户单机场景可接受（设计已知、§6 注明私钥 HMAC 为 deferred），但不能让「✅ 有 owner 闸」掩盖「非加密=防摩擦不防恶意同身份伪造」。
- **正解**：多用户上线前引入 owner 持私钥的 HMAC（设计已留位）。

A7. 🟡 其余「名不副实」小陷阱

- `handoff dump --kill-spawned`（`dump.py:1820`）不杀任何 tab——只打印清单 + 发通知（`dump.py:1615`）。名字暗示自动化，实为手动关。
- `old_ready` 写失败静默（`dump.py:948`）——evidence 通过但 `old_ready` 写不出时只一个 ⚠️ `print`，`.uri` 照发 → 继任 §0 审计无法核前任、retro 链对该棒断裂而无人察觉（自动接续下没人读 `print`）。
- `coord-identity-diag.log` 满 5MB 静默停写、不轮转、错误吞掉——恰在你最想 debug 的高负载时变瞎。
- `--no-dedupe` 是已废弃的「接受即忽略」no-op（`dump.py:1845`）。

B. 运维 / NFR 缺口（内网 CLI 工具适用维度）

商业 NFR（支付/GDPR/DSAR/WCAG/SOC2/i18n）= **N/A**（单用户单机内网工具，无网络监听、无资金、无第三方 PII）。

B1. 🔴 状态持久化：24GB 运行时状态零应用级备份

- 每次写都是 crash-atomic（temp+fsync+os.replace + fsync 父目录）、flock 内核级——单次写的持久性扎实。但整个 `~/.claude-handoff`（约 24GB）无应用级备份、无 `.git`、无 `export/restore`，只有 `Time-Machine-Included`（掩盖了「无恢复故事」这一事实）。

- **后果**：在没有最新 TM 的情况下丢失该目录 → 在飞 queue/.uri、审计 evidence、succession token 不可恢复。
- **正解**：加一个轻量周期 export（关键 sidecar + audits/ 打包）或把 `~/ .claude-handoff` 纳入显式备份；至少文档化「这是单点、靠 TM」。

B2. 🟡 哨兵无界累积 = 24GB 之源

- 实查 1804 个 ack 文件、52 个 drift marker；worktree GC 仅手动 dry-run；`prune.py` 只对账 queue 的 done/blocked，不清陈旧 ack/哨兵。
- **正解**：扩 `prune` 覆盖陈旧 ack/sidecar，或给 GC 上低频排程（叠 A2）。

B3. 🟡 凭据 / 解锁面（最敏感，但守住了红线）

- 解锁路径 = CGEvent HID 注入 **Mac 登录密码**，密码从 **Keychain** `mindpersist-login-password` 读（live 核实该 genp item 存在），**从不落本仓磁盘**（grep src/ 只有临时 0600/120s 的 succession token）。handoff-fanout shell-out 到 MindPersist 的 venv 而非 vendoring，且移除了 stray-env 启用器（只 per-project `unlock.enabled` sentinel 能 arm）。**owner 红线守住**：无 Live/DB/root/SSH 凭据下放给 AI；最后的活步骤（sync / audit-over-ride）留人类手按。
- ⚠️ **诚实标注**：「默认 OFF」的解锁安全叙事有误导——`~/ .claude-handoff/erp-system/unlock.enabled` 存在（实查），即**密码注入在 erp-system 项目已 armed**（那是 erp 链 owner 的决策，非本链）。爆炸半径 = 本机登录密码，OS-Keychain 保护。

B4. 🟡 可观测性：有日志无指标，且关键诊断会静默自盲

- router.log / auto-continue.log / watchdog.log / heartbeat 都在写；但 `coord-identity-diag.log` 5MB 上限静默停写（B in A7）= 高负载时盲。无 metrics/traces 聚合（status_board 是只读 S5a、未接进运行引擎）。
- **正解**：诊断 log 加轮转；若要长期运营，考虑把 status_board 接成真看板。

B5. ✅/🟡 失败模式总体 FAIL-OPEN、并发总体安全（但有两个真 P1，见 §C）

- 外部 CLI（codex/gemini/osascript/winlist/vscode-spaces.py）缺失或变更**只降级不损坏**（fail-open / env 可覆盖）。Quartz/winlist 探针不可用 → liveness None → fail-safe 不清。spawn_lock / atomic 内核级。**但**：spawn_lock 在慢 fs 下有 stale-break 竞态（C2），回程 helper 无 wall-clock timeout（C1）。

B6. ✅ 测试/CI 强；依赖面干净

- 67 测试文件 / 1459 `def test_` 函数 (pytest 展开 1664 cases) / 3 OS × 3 Python 真矩阵 + ruff 精确 pin + build job。Gap (势所必然)：真屏解锁/真跨桌面 focus-jump/真 osascript Enter 在 CI 跑不了 (无 GUI/无锁) → 最高风险行为靠 crafted-window + 人工 live E2E。Python 零运行时依赖；跨仓 shell-out CLI (winlist/vscode-spaces.py 住 dharmaxis) 是无 manifest 的运行时耦合，但每次调用 fail-open。

C. 已知工程加固债 (2 个 P1，独立外脑复审揪出，尚未修)

与刚闭环的派窗 focusjump 修复文件不重叠 (可并行修 / 独立 Fixer)。

C1. 🚫 回程 HELPER 无 WALL-CLOCK TIMEOUT → — HANG 冻死整个看门狗迭代

- 位置：`install/auto-continue.sh` `_return_precapture` (≈ 1326) / `_return_jump_back` (≈ 1351) 同步调 `/usr/bin/python3 vscode-spaces.py ...`，未用既有 `run_with_timeout` (`auto-continue.sh:1079-1096`，lock-probe 路径已用它)。
- 更尖的根因：被调的 `vscode-spaces.py` 无整体 wall-clock 上限；其 `ensure_winlist()` 的 `swiftc` 编译无 timeout，首跑/重编可 hang。hang 坐在 dispatch 迭代的同步主路 → 该迭代释放 per-task 锁 + 停 caffeinate 的 RETURN trap 永不执行 → mutex 占住、caffeinate 不释放、后续 spawn 阻塞。`spawn-return` 的「always exit 0」fail-open 仅在进程到达 exit 才成立，swiftc/GUI hang 直接击穿 (winlist 当前已编译=happy path 是运气非结构安全)。
- 修：两处用 `run_with_timeout ${HANDOFF_RETURN_TIMEOUT:-20}` 包裹，rc=124 → disarm+WARN+继续 cleanup。⚠️ 改 `install/auto-continue.sh` 必防 post-commit 部署陷阱 (commit 前 `export HANDOFF_INSTALL_SH=/nonexistent`，commit 后验 launcher sha 未变)。

C2. 🚫 PROJECT_SPAWN_LOCK MTIME 不刷新 → 慢 FS 下 STALE-BREAK 并发竞态

- 位置：`spawn_lock.py` `lock-dir mtime` 仅在 `mkdir()` (`spawn_lock.py:47`) 设、临界区内无 heartbeat 刷新；waiter 算 `age = now - mtime` (`:53`)、`age ≥ ttl(120s)` (`:30/:59`) 即 stale-break (`:67-79`，纯 age、无 PID 探活)。worktree caller 故意 `wait=120.0` (`spawn.py:393`) → waiter 恰在边界蹲守。
- 后果：`create_worktree` 含网络 `git fetch` (30s) + `git worktree add` (60s) + 额外 git，慢 remote 下可 >120s → 耐心的 worker #2 `rmdir` 掉仍被持有的锁、进自己临界区 → 两个并发同仓 `.git` index 改动 (正是锁要防的)。叠加：原持有者 `finally` 的 `rmdir` (`:84`) 会 cross-delete worker #2 刚建的锁。败坏了刚做的 dump-lock 对称加固本意。

- 修：持锁期 `os.utime` heartbeat / break 前加 PID 探活 / 或 `wait < ttl` / 或换 `atomic.acquire_dir_lock` (flock, 内核级) + 回归测试。

D. 文档与代码漂移（非技术读者据此易误判系统）

-  `docs/PROTOCOL.md` (289 行, schema 2) 零 retro/precheck/evidence/mandate/exit-code/old_ready/locks/worktree/singlepane/coordinator 内容——只剩 fan-out/Gate-A/原子性 base 层还对应代码。据它建「conforming 实现」会漏掉整个 v5.4 闸层。
-  `docs/ARCHITECTURE.md` 只描述原始 5 层防御，零提 unlock-pivot / 审计闸 / coordinator-succession / singlepane / worktree-isolation / winlist——现只覆盖约 1/3 的系统。
-  多处陈旧注释把已上线说成休眠：`retro_gate.py:670-672` + `auto-continue.sh:2113-2116` 写 audit-overdue producer 「deferred / dormant-but-ready」，但 producer 已 wired (`codex_audit.py:2568`)、扫描器已跑、闸已读。读注释者会误判 codex bypass 是免费午餐。
-  `codex_audit.py:7-9` docstring 写「mandate OFF (Phase A 唯一状态)」，与 `templates.py` 的「mandate ON (flipped)」同仓自相矛盾（实际代码按 mandate-ON 跑）。
-  引为 SOT 的 `codex-audit-gate-spec-draft.md` 在 HEAD 不存在 (`codex_audit.py:21` / `templates.py:181,338` 悬空引用)。
-  无项目级 `CLAUDE.md`——项目专属业务红线/领域规则未沉淀进仓（仅靠全局 + memory）。
- **bypass 机制订正**（实地坐实）：当前 6 笔 codex re-audit 债是 **PUSH 闸** `audits/bypasses/*.json`（一次性已 used 的另一机制），非 overdue 扫描器读的 `ack/*.audit.override.json`（实查盘上 0 个后者、0 个 `*.audit_overdue.txt`）→ **当前不拦 dump/交棒**。但 producer 链是 LIVE 的：未来 audit-close 用 `codex_unavailable_bypass` mode 会真写 override.json → 那笔逾期会被扫描器拦。

E. 待 owner 在环的 2 步（owner 休息中、延后）

1. **真机 E2E focus-jump**（自动派生路径）：先 `$E`-前置 GC 收敛 → 从中枢窗忘传 `--self-task` 派 worker → 验 `router.log` `focus-jump: ` + worker 落中枢桌面 + `auto-continue.log` `RETURN-REACTIVATE-WS` 回 origin。会切桌面，需 owner 看。
2. **GC --execute**（高危-owner 在环）：`handoff gc-singlepane --project handoff-fanout --protect <当前中枢> --retention-days 0 --execute`（quarantine 可逆、已限规范路径=破坏面已锁），把 active 收敛到唯一，激活 A1 自动派生。

F. 缺口优先级排序（按爆炸半径，给 owner 一眼判断）

#	缺口	严重度	爆炸半径	现状
1	install.sh 反向卸载 live 0.6.0 扩展 (A3)	高	一跑 install 就拆掉单栏/\$6c/关窗	待改 install.sh 第 5 步
2	24GB 运行时状态零应用级备份 (B1)	高	丢目录=在飞棒/审计证据/token 不可恢复	待加 export 或纳入备份
3	回程 helper 无 timeout (C1)	中-高	一 hang 冻死看门狗迭代、阻塞后续 spawn	P1 待修 (run_with_timeout 包裹)
4	spawn_lock stale-break 竞态 (C2)	中	慢 fs 下并发同仓 git 改动	P1 待修 (flock/heartbeat)
5	去程自动派生未激活 + GC 无排程 (A1/A2)	中	中枢忘传 —self-task 则 worker 不去程 (owner 主诉)	待 GC —execute + E2E (owner 在环)
6	文档覆盖 1/3、陈旧注释/悬空 SOT (D)	中	非技术读者/新会话误判系统边界	待重写 ARCHITECTURE/PROTOCOL + 修注释
7	\$6c 从未 E2E 验证 (A4)	低-中	声称 LIVE 实为 ready-idle	待一次真 E2E 留证
8	哨兵无界累积 (B2) / diag log 自盲 (B4) / owner_ack 非加密 (A6) / 名不副实小陷阱 (A7)	低	渐进/局部	待清理/排程/文档化

底线判断：功能主干 live 且经测试/日志/磁盘证据支撑，可继续日常运营；但「长期可安全自托管」需先补 #1 (install 自毁)、#2 (备份) 两块高爆炸半径，再清 #3/#4 两个 P1 加固债。其余为渐进改善，不阻断当前运营。